SQA Mid Project Report.docx

AUT University

Document Details

Submission ID
trn:oid:::20437:149899938

Submission Date
Aug 30, 2026, 9:10 PM GMT+12

Download Date
Aug 30, 2026, 9:11 PM GMT+12

File Name
SQA Mid Project Report.docx

File Size
28.7 KB

13 Pages

4,105 Words

25,096 Characters

3% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

15 Not Cited or Quoted 3%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
1% Publications
2% Submitted works (Student Papers)

Match Groups

- **15** Not Cited or Quoted 3%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
1% Publications
2% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2026-07-02 <1%
2	Submitted works	University of New York Tirana on 2026-04-10 <1%
3	Submitted works	Embry Riddle Aeronautical University on 2026-02-18 <1%
4	Submitted works	RMIT University on 2026-05-24 <1%
5	Internet	mdpi-res.com <1%
6	Submitted works	Universidad TecMilenio on 2025-10-01 <1%
7	Submitted works	Purdue University on 2026-06-28 <1%
8	Submitted works	University of York on 2010-09-14 <1%
9	Submitted works	Liverpool Hope on 2026-05-01 <1%
10	Submitted works	Purdue University on 2026-06-28 <1%

11 Submitted works

QEC on 2026-08-12 <1%

Task 1: Problem Definition and Approval
Define a meaningful real-world software quality problem
Embedded computer vision (CV) systems, used in autonomous robots, drones, and smart cameras, combine resource-strained hardware (microcontrollers, single-board computers) with computationally intensive vision models (YOLOv8, MediaPipe) to make real-time decisions. Quality Assurance for these systems focus almost exclusively on functional accuracy – does the model detect and classify objects correctly? Non-functional requirements such as latency, frame-rate stability, throughput under load, and resource usage are under-tested or ignored. This is a big gap: a detection is accurate but late, a system that degrades under load, is a failure in real-time context.
Explain the background, motivation, stakeholders, users, and practical relevance of the selected problem
Background
Real time embedded CV is common in robotics, autonomous navigation, and drones, where timing and resource constraints matter as much as accuracy. A model that performs well in benchmarks can fail in deployment if it can't hold frame rate on constrained hardware or degrades unpredictably under load. This problem is motivated by exposure this disconnect between "the model works" and "the system meets its real-time requirements"
Stakeholders and Users
Developers/Engineers – Build embedded CV pipelines and need confidence that functionality doesn't come at the cost of real-time performance

QA/Test engineers – Need a repeatable way to test NFRs alongside FRs

End users – Affected if the system misses detections due to latency or degrades under loads

Safety/compliance reviewers – operate where late or missed detections have real consequences

Practical Relevance
This problem is common across robotics. Any domain running CV on embedded hardware with real-time constraints will encounter it. Addressing it produces a testing approach usable past a single implementation.
Identify the key software quality issues involved in the problem
Key Software Quality Issues
Absence of clearly defined, testable NFRs for real-time CV systems

Lack of methods to measure and validate latency, framerate, and resource usage.

Difficulty differentiating between "the model is accurate" from "the system meets the real time requirements"

Risk of performance degradation under load

Absence of defect management processes specifically for NFR
Why the problem is suitable for an SQA Project
The problem centres on requirements quality (defining testable NFRs), test strategy design (functional vs non-functional approaches), defect management (tracking NFR violations differently from functional bugs),

and quality metrics (latency thresholds, degradation curves) which all are core SQA requirements. It also supports a requirements traceability matrix, since both functional and non-functional requirements can be trace to measurable test cases.
Feasible within a Semester
The project is a software-simulated prototype rather than physical embedded hardware, removing cost and hardware-access constraints. By using pre-recorded video/image datasets, an already existing lightweight CV mode, and simulated resource constraints, we can achieve the technical build within a few weeks, leaving the rest of the semester for QA artifacts (requirements analysis, test strategy, test cases, traceability matrix, defect management and metrics) which is where the assessment is mostly centred around.

Task 2: Requirements and Quality Analysis
Initial Functional Requirements

ID	Requirement
FR-01	The system shall load and run a pre-tainted object detection model (e.g., YOLOv8n or MediaPipe) on input video/image data.
FR-02	The system shall detect and classify objects present in each processed frame.
FR-03	The system shall output bounding boxes, class labels, and confidence scores for each detected object.
FR-04	The system shall process video input from a pre-recorded file or image sequence, simulating a live feed.
FR-05	The system shall log each frame's detection results and associated timestamp for later analysis.
FR-06	The system shall allow configuration of a simulated resource constraint (e.g., limited CPU threads, memory cap) before a test run.
FR-07	The system shall generate a summary report of detection accuracy and performance metrics after a test run completes.
FR-08	The system shall allow a user to select between at least two load conditions (e.g., baseline vs high-load) for a given test run

Initial non-functional requirements

ID	Requirement	Quality attribute
NFR-01	The system should process frames quickly.	Performance
NFR-02	The system should maintain a stable frame rate.	Performance and Reliability

NFR-03
The system should not use to much memory.
Resource Efficiency
NFR-04
The system should keep working under high load.
Reliability
NFR-05
The system should be easy to configure for different test scenarios.
Usability
NFR-06
The system should produce consistent results across repeated runs.
Reliability
NFR-07
The system should be portable across different simulated hardware profiles.
Portability
Requirement Analysis
Clarity
FR-01 to FR-08 are clear and unambiguous, using explicit action verbs("shall load"). NFR-01 to NFR-04, by contrast, rely on vague qualifiers (“quickly”, “too much”, “stable”, “keep working”), leaving them open to subjective interpretation and lacking numerical parameters.
Completeness
The FRs cover core detection logic and flow but are incomplete around error handling with no defined behaviour existing for a missing model file, corrupted frame, or invalid input stream.
Consistency
No conflicts were found, but FR-06 (resource constraint) and NFR-03(memory usage) are conceptually related but not linked.
Correctness
Requirements reflect the FR/NFR imbalance defined in Task 1; none misrepresent intended system behaviour
Feasibility
All FRs are verifiable within a software-simulated prototype. NFR-07 (portability) is feasible only if “hardware profile” means software level constraints (CPU/memory throttling).
Testability
The weakest area, because NFR-01 to NFR-05 lack numerical thresholds and cant be tested pass/fail, reflecting the core issue from Task 1: NFRs exist but aren’t stated in a testable form.
Rewritten Requirements
ID
Rewritten Requirements (Testable)
Quality attribute
NFR-01
The system should process each frame under 100ms on the hardware profile
Performance
NFR-02
The system should sustain a minimum of 15 FPS for at least 95% of its frames
Performance and Reliability
NFR-03
The system should not exceed 512MB of memory usage during a test run
Resource Efficiency

NFR-04
The system should complete detection on 100% of its input frames without crashing when under a high stress load.
Reliability
NFR-05
A user should be able to configure and do a test run (selecting resource constraint and load condition) using no more than 3 configuration steps, without having to edit the source code
Usability
NFR-06
Given similar input and configurations, the system should produce detection outputs (class labels, bounding boxes) that match roughly a 2% variance across 5 repeated runs
Reliability
NFR-07
The system should complete a full test run without fail, under at least 2 distinct simulated hardware profiles (normal and constrained CPU/Memory)
Portability
NFR-08
The system should properly handle all error conditions (missing model, corrupted frame, invalid input path) without crashing in 100% of test cases.
Reliability
NFR-09
The system shall complete model loading and first-frame inference (cold-start) in under 500ms
Performance

9
ID
Requirement
FR-06
The system shall allow configuration of a simulated resource constraint (e.g., limited CPU threads, memory cap defined NFR-03) before a test run.
FR-09
If the model file can’t be located or can’t load, the system should stop the test run and log an error message rather than failing silently
FR-10
If an input frame is corrupted or unreadable, the system should skip that frame, log the failure, and continue processing other frames
FR-11
If input video/image data is missing or the file path is invalid, the system should reject the test run before running and notify the user

FR Acceptance criteria for key features
8
Feature: Object Detection Pipeline (FR-01, FR-02, FR-03, FR-04)
Given a valid input, the system loads the model and processes every frame, producing bounding boxes, labels, and confidence scores, verifiable against manually labelled sample frames.
Feature: Detection Logging (FR-05)
Every frame creates a timestamped log, found as CSV/JSON post-run.
Feature: Resource Constraint Configuration (FR-06)
A user can select a resource constraint profile before a run, without having to edit the source code, and it is properly applied.
Feature: Load Condition Selection (FR-08)

A user can choose between load conditions before a run, and it is properly applied.
Feature: Performance Summary Report (FR-07)
A summary report is created automatically after each run.
Feature: Error Handling
Missing/invalid models stop runs with a logged error; corrupted frames are skipped and logged concurrent to processing; invalid input paths reject the run before starting.

NFR Acceptance criteria for key features
Performance (NFR-01, NFR-02)
At least 95% of processed frames are completed in under 100ms, and frame rate does not drop below 15 FPS during a baseline run.
Resource Efficiency (NFR-03)
Peak Memory usage does not exceed 512MB during a test run.
Reliability – Load (NFR-04)
Under the high load configuration, 100% of frames are processed without a crash.
Configurability (NFR-05)
A user can configure and launch a test run in no more than 3 steps, without having to edit the source code.
Consistency (NFR-06)
Repeated runs with identical input and configuration produce results within 2% variance.
Portability (NFR-07)
The system completes a full test run under at least two distinct simulated hardware profiles.
Reliability – Fault Handling (NFR-08)
All defined error conditions are handled without crashing in 100% of test cases.

Software Quality Attributes
Performance Efficiency
Processing within time/resource constraints (NFR-01, NFR-02, NFR-03).
Reliability
Maintaining function under load and faults (NFR-04, NFR-06, NFR-08, FR-09-FR11)
Usability
Ease of configuration without code changes (NFR-05, FR-06/FR-08).
Portability
Correct operation across simulated hardware profiles (NFR-07).
Maintainability
Not covered by a numbered NFR, but relevant given the project’s iterative development. Codebase should stay modular enough to swap constraint profiles, load conditions, or models without rework.
Not applicable
Security, Accessibility, Compatibility. The prototype is a local testing tool without user-facing security or accessibility requirements.
Task 3: Proposed Solution and Initial Prototype
How the Solution Addresses the Problem
The solution targets the gap from Task 1. Embedded CV systems are usually tested for functional accuracy but not real-time performance. The prototype pairs a detection pipeline with performance logging and configurable resource/load conditions, allowing functional correctness

and non-functional thresholds be tested against measurable criteria. This turns an NFR gap into something testable.

Prototype Overview

The initial prototype consists of four core modules, aligned with the FRs from Task 2:

Detection Pipeline – loads YOLOv8n and processes video/images frame by frame, producing bounding boxes, labels, and confidence scores (FR-01-FR-04).

Resource Constraint Simulator – applies software level constraints (CPU thread limiting, memory capping) to emulate embedded hardware without physical hardware (FR-06)).

Metrics Logging Module – records per-frame timestamps, detection results, latency, and resource usage(FR-05).

Summary Reporting – compiles logged data into a report with pass/fail status against NFR thresholds (FR-07)

Error handling (FR-09- FR-11) spans these modules. The pipeline validates the model on load, and the logger records skipped/corrupted frames rather than stopping the run.

Interface Justification: GUI over CLI/Dashboard

Since the solution involves configuration, workflow execution, and dashboard style output, a GUI is appropriate per the brief. The allows users to: select a resource constraint and load condition before a run (NFR-05); start a run without editing code or using a terminal; view detection results; view a performance summary with pass/fail status against NFR thresholds. This keeps the interface focused on the QA workflow (configure à run à review) whilst satisfying the GUI requirement.

Task 4: AI-Assisted Development Using GitHub Copilot

How AI-Assisted Development was used

GitHub Copilot and Claude were used throughout development to speed coding and debug unfamiliar library APIs supporting nearly every module: detection pipeline, resource constraint simulator, metrics logging, summary reporting, load condition handling, error handling, and the MAUI GUI.

Parts of the prototype supported by AI suggestions

Initial scaffolding of OnnxYoloDetector, DetectionResult, IDetector, and overall structure.

Debugging where YoloDotNet API (v4.2.0) didn’t match documentation, Copilots access to installed package IntelliSense properly identified CpuExecutionProvider, fixing errors the generic documentation search couldn’t

ResourceConstraintSimulator, MetricsLogger, SummaryReportGenerator, and LoadCondition logic

MSTest cases verifying each module

The MAUI GUI (MainPage.xaml/.cs)

Example prompts and AI-assisted outputs

Copilot identified CpuExecutionProvider() as the actual implementation class based on live access to the installed NuGet package after Claude incorrectly suggested YoloExecutionProvider.

Decompiled source inspection (VS’s Go to Definition) verified actual class/constructor signatures (YoloOptions, CpuExecutionProvider, ObjectDetection) rather than relying on assumed API shapes.

AI assisted diagnosis of a concurrency bug: inconsistent NFR-01 latency readings (890ms-3001ms) were found by isolating the test and reproducing

the bug, to MSTest's parallel execution allowing ResourceConstraintSimulators process wide CPU affinity change leak into unrelated tests fixed via [assembly: DoNotParallelize]
Limitations, errors, and risks found in AI-Assisted code
Initial Claude generated code assumed an outdated YoloDotNet API (ImageSharp) that didn't match the installed version (v4.2.0 uses SkiaSharp), needing rounds of correction against decompiled source
AI suggested code cause a cross-thread concurrency bug not caught until the inconsistent test results needed investigation. A reminder that AI-suggested needs testing under real conditions, not just in isolation.
Placeholder NFR thresholds set before real data existed needed revision once data was available. AI-suggested thresholds without proof should be treated as temporary.
Reflection on AI-assisted coding
AI assistance sped development, specifically with unfamiliar YoloDotNet API, where inspecting decompiled source via Copilot/IntelliSense proved more reliable than general AI suggestions. The bug revealed a limitation: AI code can pass isolated tests whilst hiding environment dependent bugs that surface under real conditions, certifying the need for verification through test runs, research of inconsistent results, and root cause investigation rather than trusting AI output.
Task 5: Initial Test Strategy and Test Planning
Scope of Testing:
In scope: The 4 core modules (detection pipeline, resource constraint simulator, metrics logging, summary reporting), error handling (FR-09-11), and functional correctness and non-functional characteristics (latency, frame rate, memory, reliability under pressure).
Out of scope: physical embedded hardware, security/accessibility testing, and platforms past MAUI Windows.
Test levels and types:
Integration Testing: the main level used so far, run against the real OnnxYoloDetector, model, and sample images to verify module interaction.
Unit Testing: enabled by DetectionPipelineRunner depending on IDetector rather than a concrete class, allowing mock detectors to isolate pipeline/logging/reporting logic from the real model.
System-level testing: manual end-to-end verification via the MAUI GUI (configure à run à review).
Regression testing: re-running the MSTest suite after changes, to catch issues like cross-test interference like in Task 4.
Non-functional testing: threshold/comparison checks asserted in tests or derived via SummaryReportGenerator.
Functional testing approach:
Tests pass real sample images to the detector and assert on detection output, frame counts, and JSON log contents. Error handling tests pass malformed inputs and check for correct exception type. Input validation blocks a batch if a required file is missing (FR-11); per-frame failures are caught, logged as skipped, and processing continues (FR-10).
Non-functional testing approach:
Latency and cold-start timing (NFR-01, NFR-09) are measured per frame and checked against thresholds in SummaryReportGenerator; constrained profile runs are checked for high latency. Reliability under load (NFR-04) confirms no frames are dropped in sequential or concurrent execution.
Memory usage (NFR-03) has a defined threshold but no implementation yet; NFR-08 currently only confirms no unhandled exception occurred, rather

than validating each specific error path. Both scoped as future work.
Entry Criteria:
Accepted FRs/NFRs from Task 2
Model file and sample images stored as expected.
EmbeddedCV.Tests builds against EmbeddedCV.Core successfully.
Exit Criteria:
All planned test cases executed and results recorded
Functional tests pass, or failures logged as defects with severity
Non-functional tests have measured value against threshold where applicable
No high-severity defects remain open
Results reflected in the traceability matrix, with coverage gaps (NFR-02-07) noted, not assumed
Test environment, tools and dependencies:
Visual Studio (development, Test Explorer); C# on .NET 9; .NET MAUI (Windows); YOLOv8n via YoloDotNet v4.2.0 on ONNX Runtime (CpuExecutionProvider); MSTest in EmbeddedCV.Tests, referencing EmbeddedCV.Core; test data under Assets/models/ and sample images under each project's respective sample-data folder; GitHub for version control.
Task 6: Initial Test Cases and Requirements Traceability

11 Test Cases
ID
Test Case
Requirements
Type
TC-01
Detector returns valid detections on known sample image (bounding box, label, confidence)
7 FR-01, FR-02, FR-03, FR-04
Functional
TC-02
Full pipeline processes multiple frames and gives valid JSON output (Detect à time à log)
FR-04, FR-05
Functional/Integration
TC-03
Constrained resource profile gives higher latency than baseline profile
FR-06, NFR-01
Non-functional
TC-04
Summary report evaluates NFR-01 (latency) as pass/fail against threshold
FR-07, NFR-01
Non-functional
TC-05
Cold-start latency measured and evaluated separately from steady state latency (NFR-09)
FR-07, NFR-09
Non-functional
TC-06
Batch processing under Baseline and HighLoad conditions both complete all frames without error
FR-08
Functional
TC-07

Detector constructor throws an error when model file is missing
FR-09
Negative/Error handling
TC-08
Pipeline stops a run before running when input file path is invalid
FR-11
Negative/Error handling
TC-09
System handles error conditions across runs without crashing (NFR-08)
FR-09, FR-10, FR-11, NFR-088
Reliability
Requirements Traceability Matrix
Requirement
Description
Test Case
FR-01
Load and run detection model
TC-01
FR-02
Detect/classify objects per frame
TC-01
FR-03
Output bounding boxes, labels, confidence
TC-01
FR-04
Process video/image input
TC-01, TC-02
FR-05
Log per frame timestamp + results
TC-02
FR-06
Configure resource constraint
TC-03
FR-07
Generate summary report with metrics
TC-04, TC-05
FR-08
Select between load conditions
TC-06
FR-09
Stop and log error on missing/invalid model
TC-07, TC-09
FR-10
Skip corrupted frame, continue processing
TC-09
FR-11
Reject run before running on invalid input path
TC-08, TC-09
NFR-01
Frame processing under 100ms (steady state)
TC-03, TC-04
NFR-08
Manage errors without crashing
TC-09

NFR-09
Cold-start latency under threshold
TC-05

NFR-02 through to NFR-07 are untested, we will evaluate them properly in the next phase since they need more structure that has not been written yet.

Why traceability matters

A traceability matrix supports QA by making test coverage verifiable rather than assumed. It shows which requirement each test case validates and exposes gaps. It also supports change management: when a requirement changes, as with NFR-09's cold-start threshold, the matrix shows which test cases need updating, without reviewing the whole suite.

Task 7: Project Progress, Risks, and Next Steps

Work summary

The group came to agree on a real-world software quality problem centred on non-functional requirement testing gaps in embedded computer vision systems. Initial functional and non-functional requirements were found and analysed for clarity, completeness, consistency, correctness, feasibility, and testability, with many NFRs rewritten to include thresholds and acceptance criteria defined for key features (Task 2). An initial prototype was made in C#/.NET9 using .NET MAUI (Windows), consisting of YOLOv8n (via YoloDotNet/ONNX Runtime), a metrics logging system, a resource constraint simulator (Baseline vs Constrained profiles), a summary report generator evaluating results against NFR thresholds, error handling, and a GUI dashboard allowing users to configure and run test scenarios (Task 3). GitHub Copilot was used in development to speed up coding and debug third party library integration issues (Task 4). Nine MSTest cases were created covering functional and non-functional requirements, and an initial requirements traceability matrix was made linking requirements to assess evidence (Task 6). Assessing the prototype gave us real data: baseline conditions met the NFR-01 latency threshold (79ms avg against <100ms), whilst constrained and high load conditions caused NFR failures (up to ~2150ms avg latency), showing the real-world quality problem the project is investigating.

Current limitations, risks, and challenges

Limited NFR Coverage: Only NFR-01, NFR-08, and NFR-09 are currently assessed. NFR-02 through NFR-07 haven't been assessed yet.

Provisional thresholds: Some NFR thresholds (NFR-05, NFR-06) were given placeholders and stay as estimates.

Small sample dataset: Testing so far has relied on only two sample images. This is enough to show functionality but not enough to prove real time processing.

Test isolation risk: A concurrency bug was found where parallel test execution allowed process-level state from one test to leak into another, this has been fixed but shows a broader risk of test interference as tests grow.

Simulated rather than physical constraints: Constraints are simulated rather than tested on embedded hardware, which won't reflect real world embedded performance.

Model licensing: The YOLOv8 model used is subject to Ultralytics commercial licensing terms.

Risk management

Risk
Likelihood
Impact
Mitigation
NFR-02 to NFR-07 remain untested
Medium
High. Leaves the traceability matrix with visible coverage gaps against the project's stated goal
Prioritise NFR-02 (frame rate) and NFR-07 (portability) first, since they reuse existing pipeline/simulator infrastructure; schedule remaining NFRs across the final sprint rather than left for the end
Provisional thresholds (NFR-05, NFR-06) are accepted without real measurement
Medium
Medium. Undermines the testability claims made in task 2 if left unvalidated
Collect real data for configuration step counts (NFR-05) and variance of repeated-runs (NFR-06) before finalising the thresholds
Smal sample dataset (two images) limits confidence in detection accuracy claims
Medium
Medium. Spot-check accuracy checks are not statistically meaningful on this small set
Expand the sample set with a wider range of frames (varied lighting, object density, occlusion) before the final testing
Test isolation issues recur as the suite grows
Low-Medium
High. Already caused a hard to reproduce concurrency bug
Resolve the conflicts between MSTestSettings.cs (Parallelize) and AssemblyInfo.cs (DoNotParallelize) before adding anymore tests; keep DoNotParallelize unless a valid reason to parallelise is confirmed
NFR-03 (memory) has a defined threshold but no implemented measurement
Medium
High. A defined but unmeasured NFR is a direct gap against the testability argument made in Task 2
Implement memory measurement (e.g. Process.Workingset64 sampling) in SummarReportGenerator before claiming NFR-03 as tested
NFR-08 check currently only confirms the run didn't throw unhandled, not that each error condition was correctly handled
Low
Medium. Risk of reporting a false pass on reliability
Extend the NFR-08 check to verify each error path (missing model, corrupted frame, invalid path) rather than marking success from completion alone

Plan for Final Project
Develop and run tests for NFR-02 - NFR-07 and fill in the coverage gap identified in the limitations for Task 6 and Task 7.
Measure the memory usage in SummaryReportGenerator so that there is a tangible check implemented in NFR-03, rather than a stipulated but unutilized threshold.
Ensure NFR-08 does not rely on merely not receiving an error; implement tests that check specifically for each stated error condition (non-existent model; corrupted frame; non-existent path).

Resolve MSTestSettings.cs / AssemblyInfo.cs parallelization conflict and confirm test isolation is still reliable for the entire test suite.
Expand sample image set from 2 to cover more scenarios for NFR-06 and detection accuracy;
Re-evaluate/validate provisional thresholds proposed for NFR-05 and NFR-06 against measured data.
Test up to the unit level using the IDetector abstraction, to abstract away pipeline, reporting, and logging logic, not involving real model:
Update traceability matrix: include coverage; reconcile vs. Total number of tests
Formalize an NFR-specific defect reporting procedure clearly distinguishes NFR violations to address the problem stated in Task 1.
Assemble all the various sections into final report, incorporating any remaining fixes from this revision;

Team AI reflection
AI (GitHub Copilot and Claude) was used throughout the project, from prototype development to test deign, and report drafting. It was reliable when grounded on something concrete, verifiable, and untrustworthy when depending solely on memory: in the case of Copilot, inspection of the actual installed package for YoloDotNet resolved some real build errors, whereas the initial scaffolding from Claude was based on an outdated API and required several levels of correction.
The most obvious flaw of the concurrency bug in Task 4 was that all the AI-proposed code passed merely in isolation but failed once subjected to true parallel execution, and it was only actual investigation that eventually led to the source of the problem. This firmly established the fact that all AI-Assisted code must be tested under actual circumstances, rather than examined simply by inspection.
Similar was done to the thresholds; the placeholder NFR values were adjusted after data collection instead of being left to AI.
AI was useful in progressing on unfamiliar APIs and first drafts, but each AI-proposed fact or line of code was a piece to be confirmed, not an answer.

Contribution Statements
Yaacoub: In this project, I selected the group project in the registration, created the GitHub, and the Tasks I completed were Task 1, Partial Task 2 (Rewritten requirements, FR Acceptance for key features, NFR Acceptance for key features, Software Quality Attributes), Task 3, Task 4, Task 6, Partial Task 7(Work summary, Limitations/Risks/Challenges) and created the initial GUI prototype.
Matthew: Regarding Task 2, requirements and quality analysis, for the suggested solution, I defined functional and non-functional requirements and analyzed each requirement according to 6 quality criteria: clarity, completeness, consistency, correctness, feasibility, and testability.
Regarding Task 5, initial test strategy and test planning, I have specified the scope of testing (and explicitly out of scope), the appropriate test levels and test types utilized in this project: integration, unit, system-level, regression, and non-functional tests, along with detailed explanation on functional and non-functional test approaches and established entry and exit criteria for test cycles, identified test environment, test tools, and required test-related dependencies (e.g. IDE, framework, detection model, etc.).

Regarding Task 7, project progress, risks, and next steps, I explained how the specified risks will be managed in the following stage (likelihood, impact, and mitigation actions per each), and proposed a realistic work plan until project completion regarding the uncovered NFRs, missing implementation issues, and test-related artifacts left to work on, and included a short team reflection about utilizing AI coding, testing, and quality assurance throughout the project.